TradeX B.Tech Capstone — Technical Defense Guide

Person 1: Systems, Security & Network Infrastructure Architect

CANDIDATE / DEFENDER: Person 1 **PRESENTATION TIME:** 5 to 7 Minutes

ENGINEERING ROLE: Systems, Security & Network Infrastructure Architect

CORE DOMAIN: Multi-Threaded Winsock Server, HTTP/1.1 REST Engine, RFC-6455 WebSocket Broadcaster, Salted SHA-256 & JWT Cryptography, Recursive-Descent JSON Parser

ASSIGNED REPOSITORY FILES:
- **backend/src/main.cpp**
- **backend/include/api/HttpServer.hpp**
- **backend/include/utils/Crypto.hpp**
- **backend/include/utils/Json.hpp**
- **backend/include/services/AuthService.hpp**
- **backend/include/models/Models.hpp (Shared Contracts)**

1. Executive Summary & Architectural Ownership

Person 1 is responsible for the foundational operating-system and transport layer of TradeX. Unlike standard web projects that rely on heavy Node.js or Python runtimes, TradeX implements its entire HTTP and WebSocket server natively in modern C++17 using the Windows Sockets API (Winsock2). Person 1 owns the socket lifecycle, multi-threaded request dispatching, protocol upgrading, salted SHA-256 password hashing, RFC-7519 JWT verification, and custom zero-dependency JSON serialization.

Core Defense Pitch: *"I architected the high-throughput network gateway and security subsystem in C++17. My code receives raw TCP streams on port 8080, reassembles fragmented HTTP frames, performs stateless cryptographic authentication with HMAC-SHA256 tokens, upgrades persistent client sockets to RFC-6455 WebSockets, and serializes all financial payloads through a custom recursive-descent JSON engine without any external library dependencies."*

2. Assigned Files & Detailed Code Responsibilities

backend/src/main.cpp (Daemon Entrypoint):
- Registers OS interrupt signals (SIGINT, SIGTERM) via signalHandler for clean thread shutdown.
- Initializes the MySQL database connection pool (4 pooled connections).
- Pre-loads all 32 equities into RAM and initializes the DSA Trie prefix tree.
- Launches the MarketSimulator background worker thread and binds HttpServer to port 8080.

backend/include/api/HttpServer.hpp (Winsock REST & WebSocket Core):
- Socket initialization: WSAStartup, socket(), setsockopt(SO_REUSEADDR), bind(), and listen().
- Multi-threaded worker model: Spawns std::thread(&HttpServer;::handleClient, this, clientSocket) for every accepted connection.
- TCP Packet Reassembly: Inspects Content-Length in header buffer and loops recv() until entire payload arrives.
- CORS Preflight: Intercepts OPTIONS requests and emits HTTP 204 No Content with permissive cross-origin headers.
- RFC-6455 WebSocket Handshake: Detects Upgrade: websocket, extracts Sec-WebSocket-Key, hashes with Magic String 258EAFA5-E914-47DA-95CA-C5AB0DC85B11, and sends HTTP 101 Switching Protocols.
- Broadcast Engine: Maintains active socket registry (std::vector<SOCKET> wsClients_ protected by std::mutex wsMutex_) to broadcast price ticks.

backend/include/utils/Crypto.hpp (Cryptographic Security Subsystem):
- Salted SHA-256: Implements RFC-6234 bitwise transformations (Ch, Maj, Σ0, Σ1, σ0, σ1) with 32-character random hex salts.

• HMAC-SHA256: Keyed-Hash Message Authentication Code using inner (ipad: 0x36) and outer (opad: 0x5C) XOR pads.
• JSON Web Tokens (JWT): RFC-7519 compliant Base64URL token generator and validator (Header.Payload.Signature).
• WebSocket Accept: Implements SHA-1 digest + Base64 encoding for client handshake validation.

backend/include/utils/Json.hpp (Header-Only Native C++ JSON Parser):
• Implements a recursive-descent parser and tokenizer without third-party dependencies.
• Strongly-typed Variant supporting Object, Array, String, Number, Boolean, and Null.
• Method dump() serializes C++ structs into JSON strings in microsecond time.

backend/include/services/AuthService.hpp (Identity & RBAC Service):
• registerUser(): Validates unique email/username, hashes password with salt, provisions ■1,00,000 wallet, generates JWT.
• loginUser(): Compares salted hashes, validates account status, and issues signed JWT.
• validateToken(): Stateless token validation inspecting timestamp expiration and user roles (USER vs ADMIN).

3. Data Structures & Algorithms Implemented by Person 1

Data Structure / Algorithm	Time Complexity	Space Complexity	Engineering Role in TradeX
Thread-Safe Dynamic Socket Registry	O(1) Add / O(N) Broadcast	O(C) where C = Active Clients	Maintains list of open WebSocket file descriptors with mutex lock guarding push/erase during client disconnects.
Recursive-Descent Parser (Json.hpp)	O(N) where N = Bytes	O(D) call stack where D = Nesting Depth	Tokenizes raw TCP string into JSON AST without allocating heavy intermediary objects.
HMAC-SHA256 Algorithm (Crypto.hpp)	O(B) where B = Blocks	O(1) Auxiliary	Computes cryptographic integrity tags for session tokens; enables stateless microsecond verification on every HTTP request.
Salted Key Derivation (Crypto.hpp)	O(1) Fixed 64 Rounds	O(1) Auxiliary (32B Salt)	Prevents rainbow table and dictionary attacks by salting user credentials before SHA-256 compression.

4. How Person 1's Backend Connects to the Frontend

• **Axios Interceptor Bridge:** When a user logs in via Login.tsx, AuthService::loginUser issues a signed JWT. The React client saves it in localStorage.getItem('tradex_token'). The Axios interceptor in frontend/src/services/api.ts automatically injects Authorization: Bearer <token> on every subsequent request.
• **WebSocket Handshake Bridge:** frontend/src/services/websocket.ts opens new WebSocket('ws://localhost:8080/ws'). HttpServer::handleWebSocketHandshake receives the HTTP upgrade request, computes Sec-WebSocket-Accept, and responds with status 101. Thereafter, every price tick generated in the C++ engine streams directly into React's MarketContext.tsx.

5. B.Tech Defense & Viva Voce Q&A; (Ready to Speak)

Q1: Why write a native C++ HTTP and WebSocket server instead of using Node.js Express or Python FastAPI?
Answer: *"Real-world high-frequency exchange architectures cannot afford the non-deterministic garbage collection pauses of Node.js V8 or the Global Interpreter Lock (GIL) overhead of Python. By writing the server in C++17 with Winsock, we gain deterministic sub-microsecond latency, contiguous memory layout control, and multi-threading where each worker thread processes socket I/O without runtime virtualization."*

Q2: How does your server handle TCP fragmentation over Winsock?
Answer: *"TCP is a byte-stream protocol, meaning large JSON payloads can arrive split across multiple packets. In HttpServer::parseRequest, my code inspects the initial header buffer to parse the Content-Length header. It then executes an accumulation loop calling recv() until the total bytes in memory match header_length + content_length before triggering the JSON parser. This prevents malformed payload errors under heavy network traffic."*

Q3: Explain the math and steps in your RFC-6455 WebSocket Handshake implementation.
Answer: *"The browser sends a 24-byte base64-encoded random key in the Sec-WebSocket-Key header. Per RFC-6455 specifications, my server appends the standard UUID Magic String 258EAFA5-E914-47DA-95CA-C5AB0DC85B11 to this key, calculates a 160-bit SHA-1 hash of the concatenated string, and base64-encodes the hash. We return this in the Sec-WebSocket-Accept header with HTTP 101 Switching Protocols to finalize the connection."*